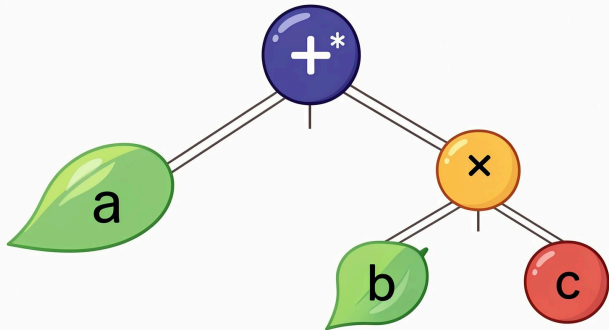


Árbol de Sintaxis Abstracta (AST): qué es y por qué importa

El AST es una representación jerárquica y abstracta de la estructura sintáctica de un programa. A diferencia del árbol sintáctico concreto, el AST omite elementos redundantes (paréntesis, tokens de formato) y conserva la semántica estructural: nodos para declaraciones, expresiones y operadores.

- Propiedades: estructura en árbol, nodos tipados, orden significativo, información mínima necesaria para análisis semántico y generación de código.
- Usos: análisis semántico, optimizaciones, generación de IR/bytecode, refactorizaciones y herramientas de IDE.





Ejemplo 1: expresión aritmética

Expresión: `a + b * c`

AST (resumido):

- Nodo '+': hijo izquierdo → Identificador 'a'
- Nodo '+': hijo derecho → Nodo '*' con hijos 'b' y 'c'

Observaciones: preserva precedencia sin nodos de paréntesis; facilita optimizaciones como reordenación o evaluación perezosa.

Ejemplo 2: declaración de función (implementación típica)

```
Fuente: func suma(x, y) { return x + y; }
```

Estructura AST:

- FunctionDecl (nombre: suma)
 - Params: [x, y]
 - Body: Return $\rightarrow$ BinaryOp '+' (x, y)

Implementación básica (pasos)

1. Definir tipos de nodo (FunctionDecl, Identifier, Return, BinaryOp, Literal).
2. Construir nodos durante el parseo (acciones semánticas del parser).
3. Proveer funciones de recorrido (visitor/traverser) para análisis y transformaciones.

- Mantener el AST independiente de detalles léxicos facilita reescritura y análisis posterior.

Generadores de analizadores y manejo de errores

Los generadores (yacc/bison, ANTLR, Menhir) producen parsers a partir de gramáticas. Un aspecto crítico es la recuperación de errores: mantener al compilador robusto y proporcionar mensajes útiles.

Panic mode (modo pánico)

Descartar tokens hasta encontrar un sincronizador (p. ej. ';' o '}'). Ventaja: simple y evita bucles infinitos. Desventaja: puede perderse contexto y producir cascada de errores.

Inserción/Eliminación de tokens

Intentar insertar el token esperado o eliminar el actual para continuar. Útil para corregir errores menores y ofrecer sugerencias automáticas.

Recuperación por producción

Usar reglas de la gramática para consumir hasta una producción completa que restaure el estado válido. Más precisa, pero requiere mayor soporte en la gramática y acciones semánticas.

Buenas prácticas para generadores:

- Definir puntos de sincronización claros en la gramática.
- Proveer mensajes de error con contexto (línea, token cercano, expectativa).
- Implementar pruebas con casos sintácticos corruptos para garantizar recuperación razonable.

❗ Ejemplo práctico: en una declaración faltante de ';', intentar inserción y continuar con modo pánico limitado para evitar pérdida de muchas declaraciones.